Data & AI Builders Conference

Building modern data foundations for intelligent solutions together

PostgreSQL: il database che semplifica la tua architettura web

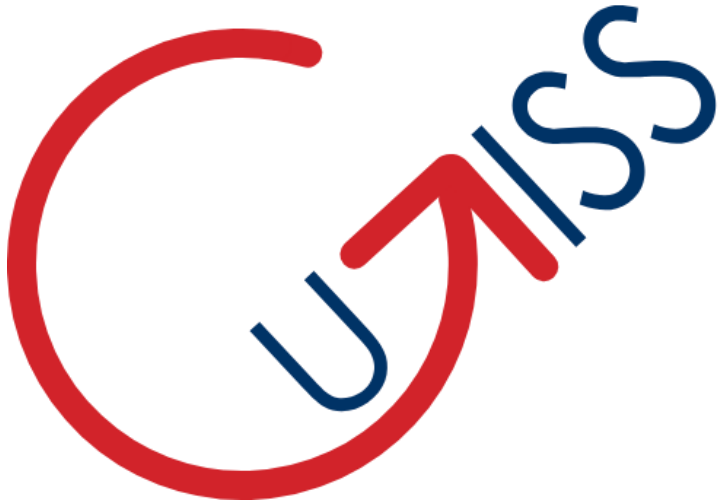
Data AI Builders · Bergamo · 8 novembre 2026



Sponsors



Collaborators





Chi sono



Danilo Dominici

Consulente indipendente

- 6x Data Platform MVP (2014-2020)
- MCT dal 2000
- Lavoro con SQL Server dalla versione 6.5
- E con PostgreSQL dalla versione 10.x
- Appassionato di AI per dare un senso ai dati

Meno componenti, meno complessità

Stack frammentato

- Database relazionale
- Database documentale
- Motore di ricerca
- Vector database dedicato
- Più sistemi da installare, integrare e mantenere

Un solo motore

- Relazionale + JSONB
- Full Text Search e pg_trgm
- pgvector per la semantic search
- Row Level Security
- LISTEN/NOTIFY per gli eventi



Agenda



- JSONB: dati documentali nel motore relazionale
- Full Text Search: ricerca testuale con ranking
- pg_trgm: similarità e ricerca fuzzy
- pgvector: semantic search
- Row Level Security: sicurezza a livello di riga
- LISTEN/NOTIFY: eventi asincroni
- Limiti da conoscere e mappa finale





01

JSONB

Dati documentali nel motore relazionale

Lodestar

json vs jsonb



json

- Copia esatta del testo in input
- Riesamina il testo a ogni esecuzione
- Conserva spazi e ordine delle chiavi
- Conserva le chiavi duplicate
- Nessun supporto agli indici

jsonb

- Formato binario decomposto
- Elaborazione molto più veloce
- Non conserva spazi né ordine delle chiavi
- Conserva solo l'ultima chiave duplicata
- Indici GIN, btree e hash
- Raccomandato dalla documentazione per la maggior parte delle applicazioni



Containment ed existence



```
SELECT '[1, 2, 3]'::jsonb @> '[1, 3]'::jsonb;        -- true
SELECT '[1, 2, 3]'::jsonb @> '[3, 1]'::jsonb;        -- true (ordine ignorato)
SELECT '["foo", "bar"]'::jsonb @> '"bar"'::jsonb;    -- true
SELECT '{"foo": "bar"}'::jsonb ? 'foo';              -- true
SELECT '{"foo": "bar"}'::jsonb ? 'bar';              -- false (solo le chiavi/elementi)

SELECT doc->'site_name' FROM websites
  WHERE doc @> '{"tags":[{"term":"paris"}, {"term":"food"}]}';
```

@> (containment) è disponibile solo per jsonb · ? controlla le chiavi di primo livello



https://www.postgresql.org/docs/current/datatype-json.html

SQL/JSON path e subscripting



```
SELECT jdoc->'guid', jdoc->'name' FROM api
  WHERE jdoc @? '$.tags[*] ? (@ == "qui")';

SELECT jdoc->'guid', jdoc->'name' FROM api
  WHERE jdoc @@ '$.tags[*] == "qui"';

SELECT ('{"a": {"b": {"c": 1}}}'::jsonb)['a']['b']['c'];
UPDATE table_name SET jsonb_field['key'] = '1';
SELECT * FROM table_name WHERE jsonb_field['key'] = '"value"';
```

Il subscripting restituisce sempre jsonb: i valori assegnati devono essere jsonb, da cui i apici



https://www.postgresql.org/docs/current/datatype-json.html

Indici GIN su JSONB



```
CREATE INDEX idxgin ON api USING GIN (jdoc);

CREATE INDEX idxginp ON api USING GIN (jdoc jsonb_path_ops);

-- indice su espressione: serve per interrogare con ? un sotto-oggetto
CREATE INDEX idxgintags ON api USING GIN ((jdoc -> 'tags'));
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc -> 'tags' ? 'qui';
```

Indice semplice: più flessibile. Su espressione: probabilmente più piccolo e veloce



https://www.postgresql.org/docs/current/datatype-json.html

jsonb_ops vs jsonb_path_ops



jsonb_ops (default)

- Operatori: ?, ?|, ?&, @>, @?, @@
- Un elemento di indice per ogni chiave e ogni valore
- Più flessibile

jsonb_path_ops

- Operatori: solo @>, @? e @@
- Un elemento con hash per valore, incluso il percorso di chiavi
- Di solito molto più piccolo e più veloce
- Contro: non può cercare se una chiave esiste; nessuna voce per strutture senza valori come {"a": {}}

https://www.postgresql.org/docs/current/datatype-json.html



02

Full Text Search

Ricerca testuale con ranking, dentro il database

Lodestar

Full Text Search: tre fasi

Parsing

- Il documento è scomposto in token: parole, numeri, indirizzi email…

Normalizzazione

- I token diventano lexemi: minuscolo, rimozione dei suffissi. Le stop word sono di solito scartate

Archiviazione

- I lexemi sono salvati in una forma ottimizzata per la ricerca, spesso con la posizione per il ranking per prossimità



tsvector, tsquery e @@



```
SELECT to_tsvector('fat cats ate fat rats') @@ to_tsquery('fat & rat');      -- true
SELECT 'fat cats ate fat rats'::tsvector @@ to_tsquery('fat & rat');         -- false

SELECT to_tsvector('fatal error') @@ to_tsquery('fatal <-> error');          -- true
SELECT to_tsvector('error is not fatal') @@ to_tsquery('fatal <-> error');    -- false

SELECT phraseto_tsquery('the cats ate the rats');
-- 'cat' <-> 'ate' <2> 'rat'
```

to_tsvector normalizza il documento, to_tsquery la query; <-> è FOLLOWED BY



Colonna generata + indice GIN



```
CREATE TABLE docs (
    id bigint PRIMARY KEY, title text, body text,
    search_vector tsvector GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(body,  '')), 'B')
    ) STORED
);
CREATE INDEX docs_search_gin ON docs USING GIN (search_vector);

SELECT id, title FROM docs
WHERE search_vector @@ websearch_to_tsquery('english', 'postgres index');
```

STORED: query semplici, ma più spazio e ricalcolo a ogni modifica



GIN vs GiST per la ricerca testuale



GIN (preferito)

- Indice invertito: una voce per lexema
- Solo colonne tsvector
- Non lossy
- Il build migliora con maintenance_work_mem

GiST

- Signature a lunghezza fissa per documento (siglen: 124 byte di default, max 2024)
- Lossy: i falsi positivi richiedono il controllo sull'heap
- Indicizza tsvector o tsquery





03

pg_trgm

Similarità, LIKE e ricerca fuzzy

Lodestar

Trigrammi e similarità



- Trigramma: tre caratteri consecutivi. "cat" → " c", " ca", "cat", "at "
- similarity(a, b): da 0 (diversi) a 1 (identici)
- % : similarità sopra pg_trgm.similarity_threshold (default 0.3)
- <% e <<% : word similarity (0.6) e strict word similarity (0.5)
- <-> : distanza = 1 − similarity()
- Modulo "trusted": installabile senza superuser con CREATE sul database



pg_trgm: indici e query



```
CREATE INDEX trgm_idx ON test_trgm USING GIN (t gin_trgm_ops);
-- oppure: USING GIST (t gist_trgm_ops)

SELECT t, similarity(t, 'word') AS sml
  FROM test_trgm WHERE t % 'word' ORDER BY sml DESC, t;

SELECT t, t <-> 'word' AS dist FROM test_trgm ORDER BY dist LIMIT 10;  -- solo GiST

SELECT * FROM test_trgm WHERE t LIKE '%foo%bar';
SELECT * FROM test_trgm WHERE t ~ '(foo|bar)';
```

Gli indici trigram accelerano anche LIKE, ILIKE, ~ e ~* senza pattern ancorato a sinistra



FTS + pg_trgm: suggerire correzioni



```
CREATE TABLE words AS SELECT word FROM
        ts_stat('SELECT to_tsvector(''simple'', bodytext) FROM documents');

CREATE INDEX words_idx ON words USING GIN (word gin_trgm_ops);
```

'simple' mantiene le parole originali. La tabella words è statica: rigenerarla





04

pgvector

Semantic search dentro PostgreSQL

Lodestar

pgvector: tipi e distanze

Tipi

- vector: float singola precisione, indicizzabile fino a 2.000 dimensioni
- halfvec: fino a 4.000
- bit: fino a 64.000
- sparsevec: fino a 1.000 elementi non zero

Operatori

- <-> Distanza euclidea (L2)
- <#> prodotto interno negativo
- <=> distanza coseno
- <+> Distanza di manhattan (L1)
- <~> Hamming, <%> Jaccard (bit)

Contesto

- Estensione open source
- CREATE EXTENSION vector;
- Ricerca esatta e approssimata
- Funziona con transazioni, replica e join



pgvector: query e indici



```
CREATE EXTENSION vector;

SELECT * FROM items ORDER BY embedding <=> '[3,1,2]' LIMIT 5;

CREATE INDEX ON items USING hnsw (embedding vector_l2_ops)
  WITH (m = 16, ef_construction = 64);
SET hnsw.ef_search = 100;       -- default 40

CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops) WITH (lists = 100);
SET ivfflat.probes = 10;        -- default 1
```

Gli esempi usano la distanza L2; per il coseno si usa vector_cosine_ops con l'operatore <=>



HNSW vs IVFFlat

HNSW

- Grafo multilivello
- Miglior compromesso velocità/recall
- Build più lento, più memoria
- Si può creare su tabella vuota

IVFFlat

- Divide i vettori in liste e cerca le più vicine
- Build più rapido, meno memoria
- Recall inferiore
- Da creare dopo aver caricato i dati
- lists ≈ righe/1000 fino a 1M righe, sqrt(righe) oltre



Filtri e ricerca ibrida

Filtrare

- Gli indici approssimati applicano i filtri dopo la scansione: possono tornare meno righe del previsto
- Indice standard sulla colonna di filtro, indice parziale, partizionamento
- Iterative scan (da 0.8.0): hnsw.iterative_scan

Ibrido con FTS

- pgvector si combina con la Full Text Search di PostgreSQL
- Unione dei risultati con Reciprocal Rank Fusion o con un reranker cross-encoder





05

Row Level Security

Chi vede quali righe, deciso dal database

Lodestar

Policy: USING e WITH CHECK



USING

- Quali righe esistenti sono visibili
- Si applica a SELECT, UPDATE, DELETE

WITH CHECK

- Quali righe nuove o modificate sono ammesse
- Si applica a INSERT e UPDATE
- Se omessa, la policy usa la stessa espressione di USING



Abilitare RLS e creare policy



```
CREATE TABLE accounts (manager text, company text, contact_email text);

ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY account_managers ON accounts TO managers
    USING (manager = current_user);

-- su users: lettura per tutti, modifica solo del proprio record
CREATE POLICY user_sel_policy ON users FOR SELECT USING (true);
CREATE POLICY user_mod_policy ON users USING (user_name = current_user);
```

Un manager non vede, modifica o cancella le righe di un altro manager



Chi bypassa RLS



- Superuser e ruoli con BYPASSRLS ignorano sempre le policy
- Il proprietario della tabella di norma bypassa RLS: ALTER TABLE … FORCE ROW LEVEL SECURITY lo include
- Policy permissive (default): combinate con OR
- Policy restrictive (AS RESTRICTIVE): combinate con AND, devono sempre passare
- TRUNCATE e REFERENCES non sono soggette a RLS



RLS: cosa tenere d'occhio



- Policy che guardano solo la riga corrente: il caso più semplice e performante
- Le espressioni di policy sono valutate prima delle condizioni utente, salvo funzioni leakproof
- Sub-SELECT nelle policy: possibili race condition (esempio in READ COMMITTED); mitigazione con FOR SHARE
- I controlli di integrità referenziale (unique, PK, FK) bypassano RLS: attenzione ai covert channel
- SET row_security = off non disattiva RLS: genera errore se una policy filtrerebbe righe





06

LISTEN / NOTIFY

Eventi asincroni senza un message broker

Lodestar

Che cos'è ?



- sistema di messaggistica asincrona nativo (Pub/Sub) che permette al database di comunicare in tempo reale con le applicazioni collegate, senza bisogno di ricorrere al polling continuo
- Use case: notifica asincrona alle applicazioni client
  - Può essere utilizzata per comunicazioni IPC (Inter-process communications)
- Costruita internamente a PostgreSQL
- Leggera
- Nessuna infrastruttura esterna
  - Niente Redis/Kafka/RabbitMQ, solo SQL



LISTEN e NOTIFY in azione



```
-- Session A

LISTEN order_watchers;

-- Session B

-- With NOTIFY command
NOTIFY order_watchers, '{"order_id": 100, "status": "completed"}’;

-- With pg_notify function
SELECT pg_notify('order_watchers', '{"order_id": 100, "status": "completed"}');
```

pg_notify(canale, payload) è più comodo di NOTIFY quando canale o payload non sono costanti



Cosa garantisce

Consegna

- Le notifiche sono consegnate dopo il commit; con rollback sono scartate
- Tra transazioni diverse: ordine di commit
- Stesso canale e stesso payload nella stessa transazione: una sola notifica

Sessione

- La registrazione LISTEN vale per la sessione e si annulla alla sua chiusura
- Solo all'interno dello stesso database
- LISTEN ha effetto al commit



Limiti di NOTIFY



Payload

- Meno di 8000 byte (configurazione di default)
- Per dati più grandi: tabella + chiave del record

Coda

- 8 GB in un'installazione standard
- Se piena, NOTIFY fallisce al commit
- Una sessione in LISTEN con transazione lunga blocca la pulizia
- pg_notification_queue_usage()

Transazioni

- Con NOTIFY o LISTEN non si può usare il two-phase commit
- Le notifiche arrivano al client solo tra una transazione e l'altra
- Connection pooling: la documentazione non ne parla, verificare il proprio pooler



Prima di aggiungere un altro sistema



- NOTIFY: payload sotto 8000 byte e coda da 8 GB sono sufficienti per il tuo caso?
- pgvector: i limiti di dimensione indicizzabile coprono i tuoi embedding?
- Ricerca vettoriale filtrata: hai verificato i risultati con indici approssimati?
- RLS: le policy sono semplici o richiedono sub-SELECT?
- FTS: la configurazione linguistica e il tipo di indice (GIN/GiST) sono scelti consapevolmente?





Recap

Lodestar

Esigenza → funzionalità PostgreSQL

Esigenza	Funzionalità	Indice tipico
Dati documentali	jsonb	GIN (jsonb_ops / jsonb_path_ops)
Ricerca testuale	tsvector / tsquery	GIN
Ricerca fuzzy, LIKE	pg_trgm	GIN / GiST (trgm_ops)
Semantic search	pgvector	HNSW / IVFFlat
Accesso per riga	Row Level Security	—
Eventi	LISTEN / NOTIFY	—



Risorse



- PostgreSQL Documentation: postgresql.org/docs/current
- JSON Types · Full Text Search · pg_trgm
- Row Security Policies · NOTIFY · LISTEN
- pgvector: github.com/pgvector/pgvector



PostgreSQL: il database che semplifica la tua architettura web

Data & AI Builders Conference

Building modern data foundations for intelligent solutions together

THANK YOU !!

Domande?

